PHASE 5 FINAL AUDIT

Read-only audit. No Kotlin, SQL, Gradle, test, configuration, or migration file was modified. No commit, push, merge, reset, or branch switch. Branch: `phase-5-onboarding`. The only file created is the audit document.

Final Verdict

PASS

Phase 5 onboarding is correctly and safely implemented. Every required behavior (1–14 below) is satisfied by code inspection and by the manual end-to-end test against the real Supabase project. The single failing unit test (`YoloPostProcessorTest`) is pre-existing teammate scan code and was not caused by Phase 5.

There is one **non-blocking** correctness gap outside the Phase 5 changeset: the repository migration history no longer matches the live Supabase `profiles` schema. This is inherited technical debt in a non-Phase-5 file and does not make the Phase 5 changeset unsafe to commit, but it must be reconciled as a follow-up.

Blocking Issues

None.

Verification of the 18 required points:

#	Requirement	Result	Evidence
1	New users cannot be marked onboarding-complete before persistence succeeds	**PASS**	`OnboardingViewModel.save()` puts `onboardingCompleted = true` only in the `UserProfile` passed to `profileRepository.saveProfile(...)` . The DB row is written by a single atomic `client.postgrest["profiles"].upsert(...)` . `saveCompleted = true` is set **only** inside `.onSuccess` ; `.onFailure` sets `errorMessage` , `isSaving = false` , leaves `saveCompleted = false` , so navigation past onboarding never fires on a failed save. No code path writes `onboarding_completed = true` independently of the full row.
2	All onboarding answers retained in `OnboardingViewModel`	**PASS**	Single `OnboardingViewModel` hoisted at `WeGlowApp` scope, shared by every onboarding composable. `OnboardingUiState` holds `fullName, ageRange, skinType, gender, isSkinSensitive` ; callbacks update via `.copy()` ; `save()` reads the whole `_uiState.value` .
3	Authenticated Supabase user ID used for persistence	**PASS**	`save()` reads `authRepository.currentUserId()` = `client.auth.currentUserOrNull()?.id` , used as `UserProfile.id` → `ProfileRow.id` → upsert primary key. `null` → error + early return, no save.
4	No user ID trusted from UI input	**PASS**	No onboarding screen callback carries an id (`onContinue(String)` , `onNext(String)` , `onAnswer(Boolean)`). `start(fullName: String?)` takes only a name. `ProfileViewModel.refresh()` / `getProfile(...)` are called only with `authRepository.currentUserId()` . RLS `with check ((select auth.uid()) = id)` is the second line of defense.
5	`onboarding_completed` is the actual source used by `hasCompletedOnboarding()`	**PASS**	`hasCompletedOnboarding = select … decodeList<ProfileRow>().isOnboardingCompleted()` , where `isOnboardingCompleted() = firstOrNull()?.onboarding_completed == true` . A row that exists with the flag `false` returns `false` . Unit-tested directly.
6	Returning completed users skip onboarding	**PASS**	`AuthViewModel.determineDestination()` → `if (hasCompletedOnboarding(userId).getOrDefault(false)) HOME else ONBOARDING` , applied at startup and post-auth. Manually confirmed: re-login to a completed account lands on Home with no onboarding.
7	Incomplete authenticated users route to onboarding	**PASS**	Same `determineDestination()` returns `ONBOARDING` when the flag is not `true` . `WeGlowApp` Loading, Login (`SignedIn` →not-HOME), and Signup all navigate to `Destination.AgeSelection` .
8	Manual signup full name preserved and persisted	**PASS**	`SignUpScreen` → `signUp` (rejects blank) → `AuthEvent.SignedUp(fullName.trim())` → `onboardingViewModel.start(event.fullName)` → `save()` writes `full_name = state.fullName.ifBlank { null }` . Manually confirmed: real name displayed instead of "Team 03".
9	Google-authenticated onboarding / name handling is safe	**PASS (by inspection)**	Google sign-in emits `AuthEvent.SignedIn` ; if incomplete, `WeGlowApp` calls `onboardingViewModel.start()` (no arg), which fully resets state and resolves the name via `authRepository.currentUserDisplayName()` reading `userMetadata` keys `full_name` → `name` → `user_name` → `preferred_username` , first non-blank trimmed, else `null` . No fabricated name; id still from `currentUserId()` . Not exercised in the manual E2E (email only).
10	Home and Profile use persisted profile name, not hardcoded "Team 03"	**PASS**	`HomeScreen` : `displayName?.let { "Hello, $it" } ?: "Hello there"` . `ProfileScreen` : `displayName ?: "Your Profile"` . `ProfileViewModel.refresh()` → `getProfile(userId).getOrNull()?.fullName` ?: `currentUserDisplayName()` . Manually confirmed.
11	Profile read-back stays behind `ProfileRepository`	**PASS**	`getProfile(userId): Result<UserProfile?>` is on the `ProfileRepository` domain interface, implemented only in `SupabaseProfileRepository` . `ProfileViewModel` depends on the interface.

12	No Supabase implementation leaked into UI / domain / feature	**PASS**	`io.github.jan.supabase` and `kotlinx.serialization.json` appear only under `data/`. Domain and feature files import only domain interfaces / androidx.lifecycle / coroutines. Screens gained only a `String?` parameter. `architectureCheck` passes.
13	No unrelated Phase 1–4 or teammate scan / ONNX / backend work changed	**PASS**	`git status` shows zero changes under `data/scan/`, `assets/acne/`, `backend/`, no `*.onnx`, no `build.gradle*`, no `libs.versions.toml`, no manifest, no `AppContainer.kt` / `ViewModelFactory.kt` / `Destination.kt`. `WeGlowApp.kt` diff limited to onboarding entry + Home/Profile name wiring.
14	Phase 5 tests adequately cover required behavior	**PASS**	22 Phase-5 assertions across `OnboardingViewModelTest` (7), `SupabaseProfileRepositoryTest` (6), `AuthViewModelTest` (+1), `ArchitectureFoundationTest` (+1). See Architecture / Diff sections.
15	Git diff review — accidental / unrelated Phase 5 modifications	**PASS**	Every hunk is Phase-5-purposeful. See "Git Diff Review".
16	Migration vs live schema	**FAIL**	Repo migration set does not represent live. See "Database Migration Consistency".
17	Mismatch to correct before committing	Identified	Migration reconciliation + RLS confirmation (follow-up, non-blocking for the code commit).
18	`YoloPostProcessorTest` failure attribution	Not caused by Phase 5	See "Remaining Test Issue".

Non-Blocking Issues

1. **Repo migration history diverges from the live `profiles` schema.** Live contains `date_of_birth` and `profile_image_url`, which no repository migration creates. `20260909000000_create_profiles.sql` describes a table that does not match how the live table was actually built (live originally had `id, full_name, gender, date_of_birth, profile_image_url, created_at, updated_at` and lacked `age_range, skin_type, is_skin_sensitive`). The live additions of `age_range`, `skin_type`, `is_skin_sensitive` were applied ad-hoc and have no dedicated incremental migration. `supabase db reset` from the repo migrations would **not** reproduce production. Inherited debt — not introduced by Phase 5; `20260909000000` is not a Phase 5 file and must not be edited in this task.
2. **RLS status on the live `profiles` table is unverified from here.** `20260909000000_create_profiles.sql` defines correct per-user `SELECT` / `INSERT` / `UPDATE` policies and `enable row level security`, but because the live table's creation history differs, confirm in the Supabase dashboard that RLS is enabled and those policies exist. The app relies on RLS as the server-side guarantee that a user can only read/write their own row (point 4).
3. **`determineDestination()` uses `hasCompletedOnboarding(...).getOrDefault(false)`.** A transient query failure routes an already-completed user back through onboarding; the re-answer performs an idempotent upsert (`onboarding_completed` stays `true`), so no data corruption, only a UX regression. Pre-existing Phase 4 pattern, retained unchanged.
4. **Double-submit window on the final Yes/No.** `isSaving` is set inside the launched coroutine, so two very fast taps can enqueue two `save()` calls before the buttons disable. Both perform the same idempotent upsert for the same authenticated id with the same data; no completion/persistence correctness impact.
5. **Google OAuth onboarding path not covered by the manual E2E.** The manual test used email signup only. The Google path is correct by inspection (point 9) but not exercised end-to-end.
6. **`ProfileViewModel.refresh()` runs on every Home/Profile entry** via `LaunchedEffect(Unit)`, i.e. one network read each time either tab is opened. Functionally fine; minor.
7. **`ProfileRow` does not model `date_of_birth` or `profile_image_url`.** Runtime-safe (Supabase's `Json` uses `ignoreUnknownKeys = true`; decode via `decodeList<ProfileRow>()`), and `profile_image_url` is intentionally deferred to Phase 6, but the code model and the live table are not 1:1.
8. **Trailing newline absent** in `AuthRepository.kt`, `SupabaseAuthRepository.kt`, `AuthViewModelTest.kt`. Pre-existing style in those files; Phase 5 edits did not add or remove it. Cosmetic.

Database Migration Consistency

FAIL.

Exact reasoning:

- **The Phase 5 migration itself is correct.** `supabase/migrations/20260910000000_add_onboarding_completed.sql` performs `alter table public.profiles add column if not exists onboarding_completed boolean not null default false;` followed by a conditional, idempotent backfill (`update … set onboarding_completed = true where onboarding_completed = false and age_range is not null and gender is not null and is_skin_sensitive is not null`). Safe, non-destructive, and accurately represents the one column added to live for Phase 5. `onboarding_completed = true` was manually confirmed present in the live table.
- **The repository migration set as a whole does not represent the live schema.**
  - Live `profiles` contains `date_of_birth` and `profile_image_url`. **No repository migration creates either column.** A clean apply of the repo migrations yields a `profiles` table missing both.

    - `20260909000000_create_profiles.sql` creates `id, full_name, age_range, skin_type, gender, is_skin_sensitive, created_at, updated_at`. The live table, before Phase 5, was `id, full_name, gender, date_of_birth, profile_image_url, created_at, updated_at`. The column sets differ in both directions, so this migration was evidently **not** the source of the live table.
    - The live additions of `age_range`, `skin_type`, `is_skin_sensitive` exist only inside the `create` migration that live did not use; there is **no incremental `ALTER TABLE … ADD COLUMN`** migration recording those changes as applied to live.
- **Consequence:** `supabase db reset` / applying the committed migrations to a fresh database produces a schema that is not production. Phase 6 (which needs `profile_image_url`) and any new environment would be built on a divergent baseline.
- **What must be corrected (follow-up, does not block the Phase 5 code commit):**
    1. Add a new forward migration bringing a clean database to parity with live — either explicit `alter table public.profiles add column if not exists date_of_birth date; add column if not exists profile_image_url text;` (least disruptive, non-destructive), or regenerate a baseline with `supabase db pull` and reconcile the migration folder.
    2. Confirm (dashboard) that RLS is enabled on the live `profiles` table with per-user `SELECT` / `INSERT` / `UPDATE` policies equivalent to `20260909000000_create_profiles.sql`; add a migration if any are missing.
    3. Optionally add `date_of_birth` / `profile_image_url` to `ProfileRow` when Phase 6 needs them (out of Phase 5 scope).

This divergence pre-dates Phase 5 and lives in `20260909000000_create_profiles.sql`, which this task must not modify.

Manual End-to-End Verification

Check	Result
New email account creation	Confirmed working. Supabase "Confirm email" disabled deliberately, because the required flow is immediate signup → onboarding.
New authenticated user reaches the existing onboarding questions	Confirmed.
All onboarding questions can be completed	Confirmed.
Final onboarding save succeeds	Confirmed.
Answers persisted to the real `profiles` table	Confirmed — `age_range` (text), `skin_type` (text), `is_skin_sensitive` (boolean) present in the live row.
`onboarding_completed = true` after a successful save	Confirmed in the live table.
Real user name displayed instead of hardcoded "Team 03"	Confirmed on Home and Profile.
Logout	Confirmed working.
Login back into the same completed account	Confirmed — goes directly to Home, onboarding is **not** repeated.
Completed returning user skips onboarding	Confirmed.
Profile picture upload	Intentionally **not** part of Phase 5 — deferred to Phase 6.

Live `profiles` columns after the safe, non-destructive additions: `id`, `full_name`, `gender`, `date_of_birth`, `profile_image_url`, `age_range`, `skin_type`, `is_skin_sensitive`, `onboarding_completed`, `created_at`, `updated_at`.

Architecture Verification

Boundaries remain intact.

- `architectureCheck` Gradle task **passes** (runs as a `preBuild` dependency; scans `domain/`, `feature/`, `ui/` for forbidden tokens incl. `io.github.jan.supabase`, and `domain/` also for `import android.` / `import androidx.` / `com.example.weglow.data`).
- New/modified domain files (`UserProfile`, `AuthRepository`, `ProfileRepository`) import only `kotlinx.coroutines.flow.Flow` and the domain model.
- New/modified feature files (`OnboardingViewModel`, `ProfileViewModel`) depend only on the domain interfaces plus `androidx.lifecycle` + coroutines.
- UI files received a `displayName: String?` parameter only.
- All Supabase / `kotlinx.serialization.json` usage is confined to `data/`.
- DI unchanged: `ProfileViewModel` is created in `WeGlowApp` with the same `viewModelFactory { … }` pattern as the existing five ViewModels. `AppContainer` and `ViewModelFactory` were not modified.

Data flow: `UI → ViewModel → ProfileRepository / AuthRepository → SupabaseProfileRepository / SupabaseAuthRepository → Supabase`, with no shortcut.

Git Diff Review

Branch `phase-5-onboarding`, working tree vs `HEAD`. **14 modified, 2 code files created, 1 migration created, plus docs.** No staged changes, no commits.

Modified (tracked)

File	Phase 5 purpose	Unrelated?
`domain/model/UserProfile.kt`	`+ val onboardingCompleted: Boolean = false`	No
`domain/repository/ProfileRepository.kt`	`+ getProfile(userId): Result<UserProfile?>` + doc on `hasCompletedOnboarding`	No
`domain/repository/AuthRepository.kt`	`+ fun currentUserDisplayName(): String?`	No
`data/model/ProfileRow.kt`	`+ onboarding_completed` field; `toProfileRow()` maps it; `+ fun ProfileRow.toUserProfile()`	No
`data/repository/SupabaseProfileRepository.kt`	`+ getProfile`; `hasCompletedOnboarding` now uses `isOnboardingCompleted()`; `+ internal fun List<ProfileRow>.isOnboardingCompleted()`	No
`data/repository/SupabaseAuthRepository.kt`	`+ currentUserDisplayName()` reading `userMetadata`; `+ import JsonPrimitive, contentOrNull`	No
`feature/onboarding/OnboardingViewModel.kt`	`start(fullName: String? = null)` resets state + resolves identity; saved `UserProfile` sets `onboardingCompleted = true`	No
`navigation/WeGlowApp.kt`	`+ profileViewModel` + `profileState`; `onboardingViewModel.start()` on the restored-session `ONBOARDING` branch and the post-auth `SignedIn` →not-HOME branch; `LaunchedEffect { profileViewModel.refresh() }` + `displayName =` on Home and Profile destinations	No — scan/hairstyle/discover/routines composables untouched
`ui/screens/HomeScreen.kt`	`+ displayName: String? = null`; `"Hello, Team 03"` → `displayName?.let { "Hello, $it" } ?: "Hello there"`	No
`ui/screens/ProfileScreen.kt`	`+ displayName: String? = null`; `"Team 03"` → `displayName ?: "Your Profile"`	No
`test/ArchitectureFoundationTest.kt`	`+ onboardingCompleted = true` in fixture; `+ assertEquals(true, row.onboarding_completed)`	No
`test/feature/auth/AuthViewModelTest.kt`	Fakes implement `currentUserDisplayName()` / `getProfile()`; `FakeProfileRepository` gains optional `storedProfile`; `+ restoredSession_withProfileRowButIncompleteOnboarding_routesToOnboarding`	No — all 8 original tests retained
`test/feature/onboarding/OnboardingViewModelTest.kt`	Fakes rebuilt to the Phase 4 `AuthRepository` contract + new members; `RecordingProfileRepository` supports injected failure + records attempted vs persisted; `+` 6 Phase 5 test cases. Fixes audit blocker B4 (unit-test source set previously did not compile)	No
`docs/phase-0/DATABASE_MODEL.md`	`+ 1` bullet documenting `onboarding_completed` and the new migration	No

Created (untracked)

File	Phase 5 purpose
`supabase/migrations/20260910000000_add_onboarding_completed.sql`	Additive migration for `onboarding_completed` + conditional backfill; leaves `20260909000000_create_profiles.sql` untouched
`feature/profile/ProfileViewModel.kt`	Loads the signed-in user's persisted profile for Home/Profile via domain interfaces only
`test/data/repository/SupabaseProfileRepositoryTest.kt`	Pure tests for the completion-decision rule and `ProfileRow ↔ UserProfile` mapping
`docs/phase-5/PHASE_5_AUDIT.md`, `PHASE_5_IMPLEMENTATION_REPORT.html/.pdf`, `PHASE_5_FINAL_AUDIT.md`	Documents from earlier steps — not project code

Accidental / unrelated modifications: None found. Every code hunk maps to a Phase 5 requirement. No formatting-only churn, no reordering of unrelated code, no dependency/version changes, no changes under `data/scan/`, `assets/`, `backend/`, or to Gradle/manifest/DI files.

Remaining Test Issue

`com.example.weglow.data.scan.YoloPostProcessorTest > ignoresBelowThresholdInvalidAndPaddingOnlyDetections` — `AssertionError` at `YoloPostProcessorTest.kt:37` (`assertTrue(detections.isEmpty())`). 1 of 6 in that suite; all other 33 unit tests pass.

Was it caused by Phase 5? No.

- `data/scan/YoloPostProcessor.kt` was last modified in commit `7b4b8bb` ("Show low-confidence detections without confidence labels"), a scan-feature change that deliberately altered low-confidence handling. `YoloPostProcessorTest.kt` was last modified in the earlier commit `945ec34` and was never updated to match `7b4b8bb`.
- The failure was **masked** before Phase 5: on a clean checkout the unit-test source set did not compile, because `OnboardingViewModelTest`'s fake `AuthRepository` was missing the Phase 4 members `authenticationState` and `signInWithGoogle` (audit blocker B4). With no compilation, `testDebugUnitTest` never executed any test. Stashing the Phase 5 changes reproduces the pre-existing `compileDebugUnitTestKotlin FAILED`.
- Phase 5 changed no file under `data/scan/` and nothing `YoloPostProcessor` depends on. It only made the pre-existing failure **visible** by repairing the test-module compilation.
- Per the task constraints (do not modify the YOLO/scan implementation), both files were left untouched. The fix is one line for the scan-feature owner: update the test's expectation, or reinstate the sub-threshold filter in `YoloPostProcessor.decode`.

Effect on the aggregate command: `./gradlew clean architectureCheck testDebugUnitTest lintDebug assembleDebug` exits non-zero **solely** because of this one test. `architectureCheck`, `compileDebugKotlin`, `compileDebugUnitTestKotlin`, `lintDebug` (0 errors), `assembleDebug` (APK produced), and every Phase 5 / onboarding / auth / profile / mapping test pass.

Safe to Commit Phase 5

YES.

Exact reason: Every Phase 5 code path is verified — completion is gated on a successful `saveProfile` upsert; all answers are retained in a single shared `OnboardingViewModel`; the persisted id comes only from `authRepository.currentUserId()` and no id is accepted from the UI; `hasCompletedOnboarding()` reads the real `onboarding_completed` column (a row with the flag `false` is treated as incomplete); returning-complete vs incomplete routing works via `determineDestination()`; manual-signup and Google display names are handled without fabrication and persisted; Home/Profile read the name through `ProfileRepository`; and no Supabase type crosses into `domain/`, `feature/`, or `ui/` (`architectureCheck` passes). The full new-user flow was manually confirmed end-to-end against the real Supabase project, including `onboarding_completed = true`, the real name replacing "Team 03", and a completed user skipping onboarding on re-login. The Phase 5 migration is idempotent, non-destructive, and matches what was applied to live. The only failing test is pre-existing teammate scan code not caused by Phase 5.

The `Database Migration Consistency: FAIL` finding does **not** block committing the Phase 5 changeset: the divergence (missing `date_of_birth` / `profile_image_url` migrations; `20260909000000_create_profiles.sql` not matching live's creation history; ad-hoc live `ALTER`s without incremental migration files) is inherited technical debt in a file this task must not modify. Reconcile it in a follow-up (a forward `ALTER TABLE … ADD COLUMN IF NOT EXISTS` migration or a `supabase db pull` baseline) and confirm RLS is enabled on the live `profiles` table. Until that follow-up lands, `supabase db reset` from the repo migrations will not reproduce production.